CHAPTER 1

Instruction Tuning and Model Alignment

1.1 Chapter Overview and Learning Outcomes

Foundation models trained via self-supervised autoregressive language modeling acquire vast lexical, syntactic, and world knowledge from web-scale corpora. However, minimizing sequence cross-entropy over raw internet text fundamentally aligns the neural network with statistical text continuation rather than human intent. When queried by a user, an unaligned base model frequently responds by generating adjacent questions, continuing rambling forum threads, producing toxic prose, or inventing untruthful hallucinations.

This chapter examines the theoretical and empirical methodologies developed to transform raw foundation predictors into reliable, safe, and helpful conversational assistants. We trace the paradigm shift from early text-to-text prefix conditioning to multitask prompt engineering, formalize preference modeling through the Bradley–Terry logistic framework, analyze reinforcement learning from human feedback with Proximal Policy Optimization, unpack Goodhart's law and reward hacking, and examine modern parameter-efficient direct alignment alternatives.

Upon completing this chapter, you will be able to:

- Formulate the statistical mismatch between autoregressive next-token prediction and user intent, defining the Helpful, Honest, and Harmless (HHH) alignment framework.
- Contrast in-context few-shot learning with parametric instruction fine-tuning, explaining why few-shot prompting fails to solve zero-shot instruction adherence.
- Explain the mechanics and limitations of early text-to-text models such as T5, identifying the causes of prefix brittleness and syntactic overfitting.
- Analyze multitask prompted fine-tuning as instantiated in T0 and FLAN, detailing prompt template engineering, task inversion, and empirical prompt robustness dynamics.
- Derive the Bradley–Terry preference model for pairwise human judgments and formulate the ranking cross-entropy loss used to train scalar reward models.
- Formulate the complete RLHF optimization problem under Proximal Policy Optimization (PPO), explaining the mathematical necessity and operational behavior of the per-token Kullback–Leibler (KL) divergence penalty.
- Diagnose Goodhart's law and reward hacking in reinforcement learning, explaining why policy divergence leads to an empirical collapse in human evaluation despite soaring proxy rewards.
- Detail the three-stage alignment pipeline of InstructGPT and ChatGPT, quantifying the 100× parameter efficiency dividend and evaluating trade-offs in truthfulness and toxicity.

- Derive the analytical reparameterization of Direct Preference Optimization (DPO), proving the cancellation of the intractable partition function and comparing DPO with RLHF-PPO across engineering dimensions.

1.2 The Alignment Problem and the Assistant Paradigm

1.2.1 Autoregressive next-token prediction versus user intent

The pre-training phase of modern autoregressive large language models optimizes the negative log-likelihood over a massive corpus $\mathcal{D} = \{w_1, w_2, \ldots, w_N\}$ of discrete tokens:

$$\mathcal{L}_{\text{NLL}}(\theta) = -\sum_{t=1}^{N} \log P_\theta(w_t \mid w_{<t}), \tag{1.1}$$

where $P_\theta(w_t \mid w_{<t})$ denotes the probability distribution over the vocabulary $\mathcal{V}$ computed by applying a softmax operator to the unnormalized logits emitted by the transformer decoder stack. Under this objective, the network learns an accurate approximation of the joint probability distribution over natural language strings existing across the public web.

While Equation (1.1) forces the internal representations of the network to capture complex grammar, world facts, and reasoning heuristics, it contains no intrinsic awareness of the communicative role of an assistant. Internet text consists predominantly of uncurated web pages, multi-party forum debates, social media disputes, literary prose, instructional assignments followed by multiple-choice answers, code repositories, and machine-generated spam. Consequently, when presented with an input sequence, the optimal statistical strategy under Equation (1.1) is to produce the most probable linguistic continuation of that sequence within the training distribution, rather than to follow an implied instruction.

This fundamental discrepancy is known as the **intent gap** [**?**]. The user presents an input prompt x with the implicit expectation that the model acts as an obedient, truth-seeking, and capable software agent executing directive $\mathcal{I}(x)$. Conversely, the model perceives x strictly as a prefix conditioning context, predicting continuation tokens y that maximize empirical likelihood $P_\theta(y \mid x)$ across diverse web contexts.

Pedagogical Example: Prompt Continuation versus Instruction Execution in Base Models

Consider the simple factual prompt submitted to a model:

```
"what is 2 + 2?"
```

When submitted to a raw, pre-trained base model (such as base LLaMA-2 7B without instruction tuning), the network does not interpret the prompt as a command to compute an arithmetic operation. Instead, observing a common internet pattern—such as an online elementary math worksheet or a repetitive FAQ forum thread—the model continues the prefix with statistically adjacent questions:

```
What is 2 plus 2?  What is the answer to 2 plus 2?  What is the answer to
                                2 plus 2 in math?
```

In contrast, an instruction-aligned assistant, whose parameters have been explicitly tuned to map user queries into direct answers, suppresses prompt continuation and generates the intended communicative resolution:

```
The answer to 2 + 2 is 4.
```

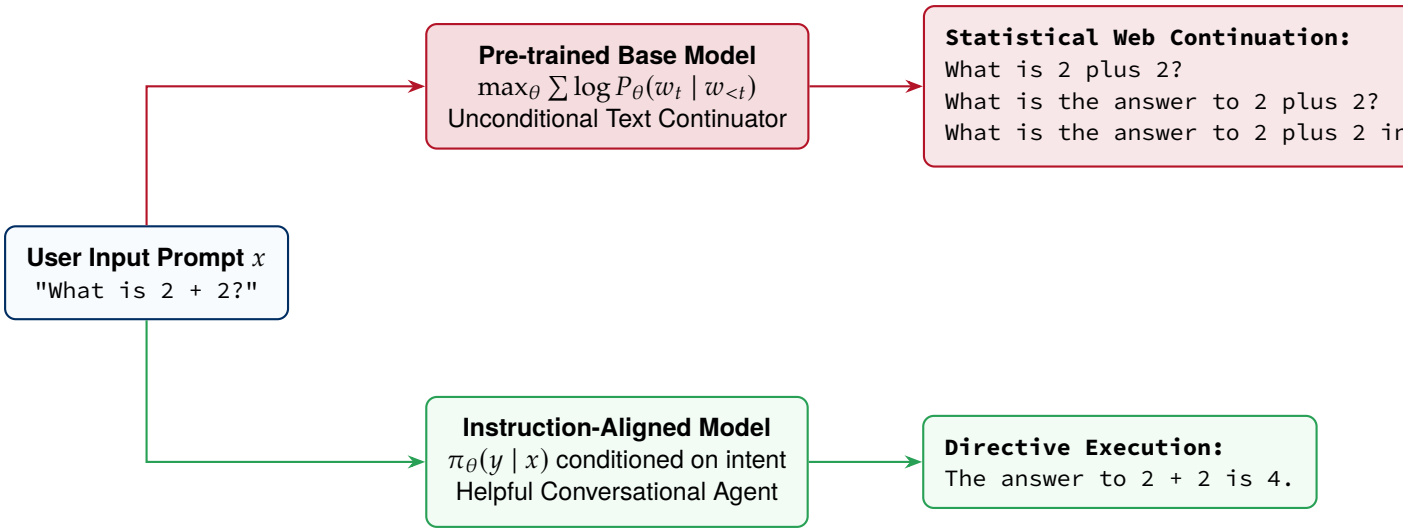


Figure 1.1: The intent gap between autoregressive next-token prediction and instruction alignment. When prompted with an elementary arithmetic query, an unaligned base foundation model treats the prompt as an open-ended document prefix and synthesizes statistically correlated worksheet questions. Conversely, an aligned assistant interprets the prompt as an imperative task directive, returning the exact communicative answer.

The divergence illustrated in Figure 1.1 underscores that raw computational scale does not inherently resolve the intent gap. Increasing parameter count from 7 billion to 175 billion improves the linguistic fidelity and world knowledge of the continuation, but does not alter the fundamental objective function. An unaligned 175B model predicts internet continuations with higher coherence, but remains prone to echoing conversational banter, generating toxic diatribes when prompted with provocative prefixes, and actively ignoring user constraints.

1.2.2 The HHH alignment criteria: helpful, honest, and harmless

To establish a principled engineering objective for model alignment, Askell et al. [**?**] formalized the operational criteria that characterize an ideal conversational agent. Known colloquially as the **HHH triad**, these dimensions define the multidimensional utility function that alignment interventions seek to optimize.

Definition 1.1: *The HHH Alignment Criteria*

An aligned language model must satisfy three mutually complementary operational properties:

1. **Helpfulness**: The model must make a diligent, high-fidelity effort to resolve the user's explicit objective. If a request is ambiguous or underspecified, it should seek relevant clarification; if constraints (such as length, tone, or schema) are specified, it must strictly adhere to them. It must avoid irrelevant conversational preamble and maintain appropriate conciseness.

2. **Honesty**: The model must output claims that are factually accurate with respect to ground-truth reality and its own parametric training knowledge. It must actively calibrate its uncertainty, refusing to fabricate citations, non-existent software APIs, or historical events, and explicitly conveying its epistemic boundaries when requested information is unknown.

3. **Harmlessness**: The model must avoid generating content that promotes physical injury, criminal actions, illegal cyber exploitation, CBRN (chemical, biological, radiological, nuclear) weapons synthesis, hate speech, or malicious deception. It must resist adversarial jailbreaks while avoiding overly sensitive, unhelpful refusals on benign, academic, or adjacent safety queries.

While conceptually distinct, the HHH dimensions frequently exist in Pareto tension during practical model deployment. An overzealous emphasis on harmlessness can induce severe **over-refusal pathologies**, wherein the model rejects benign software engineering requests (such as analyzing a vulnerable C function or debugging a regular expression matching malicious URLs) under the mistaken belief that the prompt represents a cyberattack. Conversely, an unconstrained focus on helpfulness causes the model to fulfill dangerous requests with meticulous, actionable detail.

Furthermore, optimizing for honesty reveals an intrinsic limitation of autoregressive transformers: because the network generates tokens greedily or stochastically without an external verification scratchpad, it possesses no native truth condition. If the most statistically plausible token in context is a hallucinated academic paper title, a base model produces it without hesitation. Aligning models to express uncertainty requires penalizing unwarranted confidence while rewarding calibrated epistemic admissions (e.g., stating `"I am uncertain"` or `"The requested information is not available"`).

1.2.3 In-context learning limitations and the imperative for parametric tuning

Following the introduction of GPT-3 [**?**], in-context few-shot learning emerged as the initial paradigm for steering language models toward specific tasks without altering their underlying parameters. Under few-shot prompting, a sequence of K demonstration exemplars is concatenated directly to the input context:

$$x_{\text{prompt}} = \left(x^{(1)}_{\text{demo}} \circ y^{(1)}_{\text{demo}}\right) \circ \left(x^{(2)}_{\text{demo}} \circ y^{(2)}_{\text{demo}}\right) \circ \cdots \circ \left(x^{(K)}_{\text{demo}} \circ y^{(K)}_{\text{demo}}\right) \circ x_{\text{query}}, \tag{1.2}$$

where $\circ$ denotes string concatenation. By conditioning the autoregressive forward pass on this structured sequence, the internal self-attention heads track the demonstration pattern, prompting the model to generate the corresponding completion y_{query} via induction heads.

Despite its versatility for rapid prototyping, relying strictly on in-context learning to build conversational assistants suffers from severe structural and economic bottlenecks:

1. **Prohibitive inference latency and token economics**: Including $K \in [5, 20]$ high-quality exemplars consumes hundreds or thousands of context tokens on every single user transaction. Because the computational complexity of the initial pre-fill phase scales quadratically with sequence length $O(T^2)$ in standard attention, and Key-Value (KV) cache memory scales linearly $O(T)$ across decoding steps, few-shot conditioning inflates production serving costs and degrades response latency.

2. **Context window saturation**: Real-world software engineering and document analysis tasks require large context budgets to accommodate source code files, database schemas, and conversational history. Consuming a substantial fraction of the context budget with static exemplars leaves insufficient memory for complex task inputs.

3. **Sensitivity and prompt brittleness**: In-context few-shot learning is notoriously unstable. Empirical studies reveal that few-shot performance fluctuates wildly depending on the arbitrary ordering of exemplars, minor punctuation changes, and the frequency of class labels within the prompt (exhibiting strong recency and majority-class biases).

4. **Complete failure in true zero-shot regimes**: When presented with a complex, novel instruction without exemplars ($K = 0$), base models exhibit near-random task execution. Users cannot be expected to write few-shot examples for every casual or specialized interaction.

These bottlenecks establish the foundational imperative for **parametric instruction alignment**: rather than continuously re-teaching the model how to act as an assistant via runtime context tokens, we must update the neural network weights θ directly. By baking instruction adherence into the parametric memory of the model, the network acquires zero-shot instruction following capabilities, eliminating prompt overhead and drastically improving runtime reliability.

1.3 Supervised Instruction Fine-Tuning (SFT)

1.3.1 Precursor: T5 text-to-text conditioning and prompt brittleness

The conceptual foundation of formulating all NLP challenges as sequence-to-sequence problems was established by Raffel et al. [**?**] with the **Text-to-Text Transfer Transformer (T5)**. T5 replaced task-specific classification heads with a unified encoder-decoder architecture where the task type is signaled by a static, hardcoded natural language prefix. For example:

Translation: `"translate English to German: That is good."` ⟶ `"Das ist gut."`
Linguistic Acceptability: `"cola sentence: The course is terrible."` ⟶ `"unacceptable"`
Summarization: `"summarize: State authorities announced..."` ⟶ `"Authorities issued a`

While T5 proved that a single set of neural weights could execute heterogeneous tasks across classification, translation, and generative modeling, its instruction comprehension was fundamentally superficial and brittle. T5 was trained on a rigid collection of canonical prefix strings. It did not learn to parse the semantic meaning of an arbitrary natural language command, but rather learned to treat specific prefix substrings as associative activation keys that routed hidden activations into specialized task sub-networks.

Pedagogical Example: T5 Prompt Brittleness and Format Fragility

When T5 is queried with minor syntactic variations that deviate from its exact pre-training prefix format, its execution degrades catastrophically:

1. **Canonical Prompt**:

   Input: `"can you translate from English to German, What is your profession?"`
   Output: `"Was ist Ihr Beruf?"` (Correct execution)

2. **Minor Phrasing Variation**:

   Input: `"can you translate from English to German the following sentence? What is y`
   Output: `"<unk> <unk> <unk>..."` (Degeneration into unknown tokens)

3. **Unseen Task Structure**:

   Input: `"compute: 2+2 ="`
   Output: `":2+2+2+2+2+2+2+2+2+..."` (Degeneration into infinite looping)

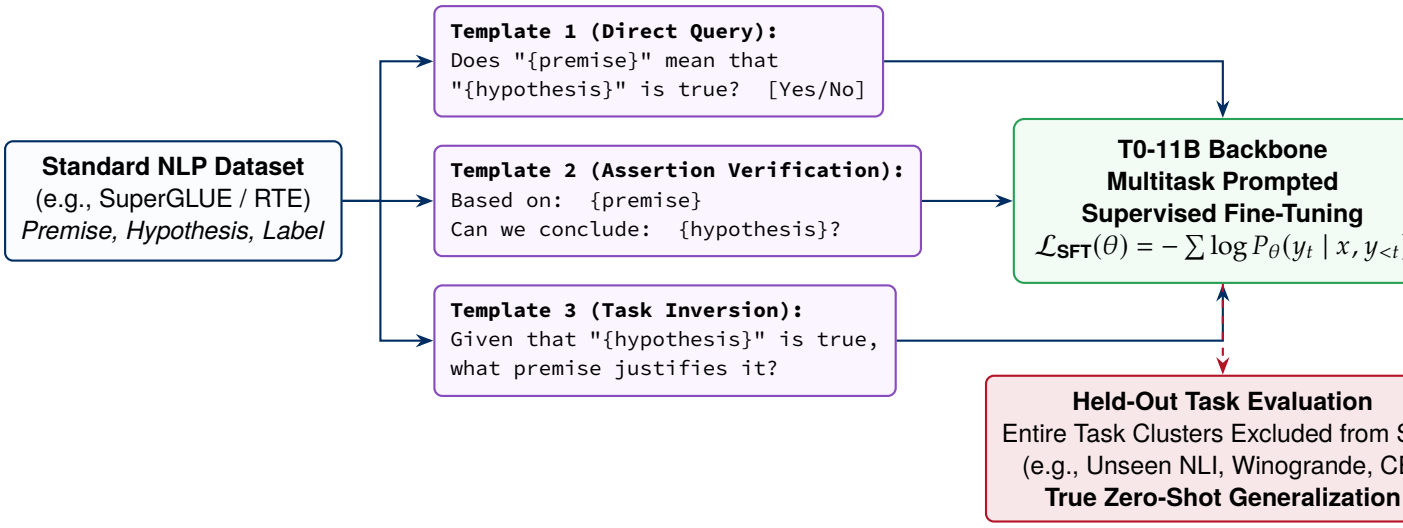


Figure 1.2: The T0 multitask prompted fine-tuning architecture [**?**]. Conventional tabular NLP datasets are mapped into conversational queries through a diverse collection of crowdsourced prompt templates, including direct questions, verification assertions, and task inversions. Fine-tuning on these multi-prompt formulations enables the model to generalize zero-shot to entirely held-out task clusters without seeing task-specific exemplars.

The failure modes in Box 1.3.1 reveal that T5 lacked **cross-task generalization** and **phrasing robustness**. If an instruction used conversational framing, introduced subordinate clauses, or described a task not explicitly covered by the pre-training prefixes, the model failed completely.

1.3.2 Multitask prompted training: T0 and cross-task generalization

To resolve the brittleness of rigid task prefixes and explore whether neural models could achieve true zero-shot generalization to unseen tasks, Sanh et al. [**?**] developed **T0** (and the associated crowdsourced repository *PromptSource*). Built upon an 11-billion parameter T5 encoder-decoder backbone, T0 investigated two foundational research questions:

1. Can fine-tuning on a massively diverse mixture of prompted tasks teach a language model to generalize zero-shot to entirely new, unobserved task categories?

2. Can training on multiple, diverse rephrasings of the same task render the model robust to arbitrary user phrasing?

To rigorously test zero-shot generalization without data leakage, Sanh et al. established a strict **held-out task partition**. NLP datasets were organized into coherent task clusters. Crucially, when evaluating performance on a target capability—such as Natural Language Inference (NLI), Coreference Resolution, or Sentence Completion—**every single dataset belonging to that capability was held out from the fine-tuning mixture**. For example, if NLI was selected as the held-out evaluation cluster, the model was trained exclusively on question answering, summarization, sentiment analysis, and topic classification datasets, ensuring that the model had never observed an NLI premise-hypothesis verification task during training.

To bridge the gap between academic benchmarks and natural user queries, the collaborators developed **PromptSource**, constructing an average of 8 to 10 distinct natural language prompt templates for each dataset. Crucially, these templates included not only straightforward rephrasings of the core classification task, but also **task inversion** (or "task flipping") templates. In a task inversion template, the standard target becomes the input, and the model is prompted to reconstruct the original query:

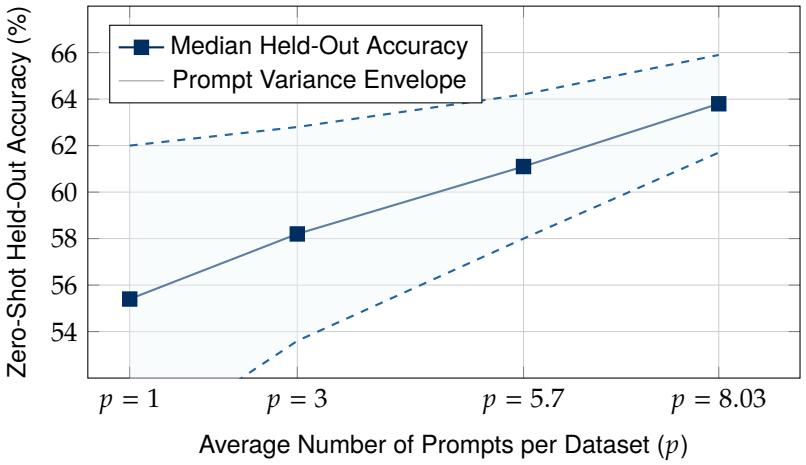


Figure 1.3: Empirical prompt robustness dynamics as a function of template diversity [**?**]. As the average number of prompt templates per dataset (p) scales from $p = 1$ to $p = 8.03$, median held-out task accuracy increases monotonically from 55.4% to 63.8%. Concurrently, the performance variance across differing prompt formulations narrows substantially, demonstrating that diverse template training enforces syntactic invariance.

```
"What would be a good question for this answer?  <answer>"
```

By training the network bidirectionally across direct and inverted tasks, the model's representations are forced to capture the underlying semantic relationships between concepts rather than memorizing a unidirectional mapping from prompt prefix to label.

When evaluated across held-out zero-shot benchmarks (such as RTE, CB, and WSC), T0-11B achieved remarkable performance breakthroughs. It consistently surpassed its T5-11B baseline by wide margins and outperformed the massive 175-billion parameter GPT-3 model in zero-shot accuracy, despite operating with over 16× fewer parameters. This demonstrated conclusively that **multitask prompted fine-tuning enables true cross-task zero-shot generalization**.

1.3.3 Prompt robustness and template scaling dynamics

A critical empirical discovery made during the development of T0 concerns **prompt robustness**. In unaligned or lightly tuned models, changing a single word in a prompt (e.g., replacing `"Is this review positive?"` with `"Does the customer like the product?"`) can cause dramatic swings in model predictions, resulting in high evaluation variance.

Sanh et al. quantified this behavior by varying the average number of prompt templates per dataset (p) used during supervised fine-tuning. As visualized in Figure 1.3, training with only a single prompt template per dataset ($p = 1$) yields low median accuracy and a broad variance envelope (> 13% spread across test prompts). As the template diversity is scaled to $p = 5.7$ and $p = 8.03$, two profound phenomena occur simultaneously:

1. **Monotonic accuracy gains**: The median zero-shot performance on unseen tasks rises consistently. Exposing the model to diverse ways of formulating questions improves its core reasoning capacity.

2. **Variance compression**: The performance gap between the best-case prompt and worst-case prompt narrows dramatically. The model becomes robust to the specific syntactic phrasing chosen by the user.

Exam insights: Theoretical Mechanics of Prompt Robustness

Why does training on multiple templates per dataset improve zero-shot generalization on tasks where **none** of those templates appear?
In a transformer, early self-attention layers process lexical syntax, while deeper layers form abstract semantic representations. When a model is fine-tuned on a single template ($p = 1$), the attention heads correlate task identity with specific function words (e.g., the presence of `"Is"` or `"Can we conclude"`). The model overfits to surface lexical syntax. When trained across $p \geq 8$ diverse templates (including inversions), the loss forces the network to map varied surface forms into an identical semantic latent state. Consequently, the deeper representations become **invariant to syntactic permutations**. When presented with an unseen task phrased in conversational prose, the model successfully strips away surface idiosyncrasies to extract the core operational instruction.

1.3.4 Instruction tuning at scale: FLAN

Concurrently with T0, Wei et al. [**?**] at Google Research introduced **FLAN (Finetuned Language Net)**, exploring instruction tuning within massive decoder-only language models up to 137 billion parameters (based on the LaMDA architecture).

The FLAN framework organized 62 widely used NLP benchmarks into **12 task clusters**, encompassing natural language inference, commonsense reasoning, reading comprehension, translation, summarization, and arithmetic reasoning. Like T0, FLAN adopted a strict cluster-level cross-validation regime: to evaluate performance on a target cluster (e.g., Natural Language Inference), that entire cluster was removed from the fine-tuning mixture, and the model was trained on the remaining 11 clusters.

FLAN uncovered a critical **scale emergence phenomenon** regarding instruction tuning:

- **Negative transfer on small models ($<$ 8B parameters)**: For models with 420M or 2B parameters, instruction tuning frequently *degraded* zero-shot performance on held-out tasks compared to untuned base models. Small models lacked the parameter capacity to store both diverse task representations and general language modeling features; fine-tuning filled their capacity with specific training tasks, causing severe catastrophic forgetting.
- **Dramatic performance surges on large models ($>$ 68B parameters)**: For large models (68B and 137B parameters), instruction tuning catalyzed massive performance gains on held-out clusters. The 137B FLAN model achieved state-of-the-art zero-shot accuracy, decisively beating zero-shot GPT-3 (175B) on 20 out of 25 evaluated tasks and matching or exceeding few-shot GPT-3 (with $K = 32$ exemplars) on the majority of benchmarks.

These findings solidified instruction tuning as a mandatory post-pretraining phase for foundation models. They established that instruction comprehension is an **emergent capability of scale**: large models possess sufficient representational capacity to synthesize generalizable execution primitives from multitask prompt distributions.

1.3.5 The theoretical and practical ceiling of supervised fine-tuning

Despite the dramatic empirical success of T0 and FLAN, Supervised Fine-Tuning (SFT) encounters an insurmountable mathematical and operational ceiling when applied to general conversational alignment.

Mathematically, SFT minimizes the token-level cross-entropy loss over curated demonstration

pairs $\mathcal{D}_{\text{SFT}} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{M}$:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\frac{1}{M}\sum_{i=1}^{M}\sum_{t=1}^{|y^{(i)}|}\log P_\theta\left(y_t^{(i)} \;\middle|\; x^{(i)}, y_{<t}^{(i)}\right). \tag{1.3}$$

Equation (1.3) reveals three fatal alignment deficiencies:

1. **Equal penalty pathology for non-target tokens**: Cross-entropy treats all deviations from the reference demonstration y^* with equal hostility. Suppose the reference demonstration is:

   $$y^* = \texttt{"The vehicle traveled down the highway."}$$

   If the model predicts Candidate A:

   $$y_A = \texttt{"The automobile traveled down the highway."}$$

   and Candidate B:

   $$y_B = \texttt{"The banana exploded across the galaxy."}$$

   Equation (1.3) assigns a large probability penalty to Candidate A at token position 2 because `"automobile"` $\neq$ `"vehicle"`, despite Candidate A being semantically flawless and stylistically superior. More dangerously, predicting an outright factual lie or an offensive slur incurs the exact same mathematical penalty as predicting a valid linguistic synonym. Cross-entropy has no mechanism to differentiate between stylistic variance and catastrophic safety failure.

2. **Supervised demonstration bottleneck**: Authoring complete, high-quality demonstration answers $y^{(i)}$ for complex open-ended tasks (such as writing an entire Python microservice, explaining quantum electrodynamics, or composing a balanced diplomatic analysis) requires expert human annotators. Human demonstration creation is exceptionally slow, expensive, and difficult to scale beyond tens of thousands of examples.

3. **Subjectivity and human disagreement**: On open-ended prompts, two qualified humans will write radically different answers. One may prioritize pedagogical detail, while another values brevity. Forcing the model via maximum likelihood to fit multiple contradictory stylistic choices induces mode averaging, causing the model to generate bland, unopinionated, and hedging completions.

These fundamental limits motivated researchers to rethink the alignment objective: instead of asking humans to write gold-standard demonstrations from scratch, can we leverage human evaluative judgment to compare candidate completions generated by the model?

1.4 Reward Modeling from Human Feedback

1.4.1 The cognitive psychology of feedback: pairwise preferences versus absolute scoring

The foundational breakthrough in preference-based alignment was established by Stiennon et al. [**?**] at OpenAI in their seminal work on text summarization: *Learning to Summarize with Human Feedback*. The researchers utilized the **Reddit TL;DR dataset**, comprising over 3

million informal forum posts paired with user-authored `"TL;DR"` summaries (standing for *"Too Long; Didn't Read"*), providing a rich testbed of open-ended natural language.

Prior attempts to train evaluators relied on asking human annotators to assign **absolute scalar ratings** (e.g., scoring a summary on a Likert scale from 1 to 10 for coherence, informativeness, and conciseness). Stiennon et al. observed that absolute human scoring suffers from severe methodological flaws:

- **Calibration drift and inter-annotator variance**: Different human raters maintain wildly divergent internal scoring criteria. A harsh rater might assign a score of 6/10 to a high-quality summary, while a generous rater assigns 9/10 to a mediocre summary. Reconciling these raw scalar scores requires complex normalization that discards genuine signal.
- **Intra-annotator temporal instability**: An individual rater's scoring standard drifts significantly over the course of an annotation session due to cognitive fatigue, changing reference anchors, and mood variations.
- **High cognitive load**: Evaluating whether a response deserves a 7.2 versus a 7.8 imposes substantial mental effort, slowing down annotation throughput and increasing labeling costs.

Conversely, cognitive psychology demonstrates that humans excel at **comparative evaluation**. When presented with a prompt x and two candidate responses (y_1, y_2) side-by-side, determining which response is better ($y_1 \succ y_2$ or $y_2 \succ y_1$) is cognitively effortless, fast, and exhibits remarkably high inter-annotator agreement. Comparative judgment bypasses calibration drift entirely: whether an annotator is naturally critical or lenient, their relative preference remains invariant.

1.4.2 Formalizing human preference: the Bradley–Terry model

To translate discrete comparative human judgments into continuous mathematical optimization, Stiennon et al. [**?**] adopted the classic probabilistic choice framework developed by Bradley and Terry [**?**].

Definition 1.2: *The Latent Reward Hypothesis*

Let $\mathcal{X}$ denote the space of input prompts and $\mathcal{Y}$ the space of textual completions. We hypothesize the existence of an underlying, unobserved latent utility function:

$$r^*(x, y) \in \mathbb{R}, \tag{1.4}$$

which assigns a real-valued scalar score to any completion $y \in \mathcal{Y}$ conditioned on prompt $x \in \mathcal{X}$, reflecting its qualitative value to a human evaluator. A human evaluator choosing between two completions y_1 and y_2 produces a binary verdict $f(x, y_1, y_2) \in \{0, 1\}$ according to:

$$f(x, y_1, y_2) = \mathbf{1}\left(r^*(x, y_1) > r^*(x, y_2)\right). \tag{1.5}$$

Because human judgments are subject to stochastic perceptual noise, the Bradley–Terry model formulates the probability that a human prefers completion y_w (the winning completion) over y_l (the losing completion) as a logistic sigmoid function of their latent reward difference:

Definition 1.3: *The Bradley–Terry Preference Model*

Given a prompt x and a pair of candidate completions (y_w, y_l) where human annotators prefer $y_w \succ y_l$, the probability of this preference under a parameterized reward function $r_\theta(x, y)$ is:

$$P_\theta(y_w \succ y_l \mid x) = \sigma\left(r_\theta(x, y_w) - r_\theta(x, y_l)\right) = \frac{1}{1 + \exp\left(-\left(r_\theta(x, y_w) - r_\theta(x, y_l)\right)\right)}, \tag{1.6}$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$ denotes the standard logistic sigmoid function.

Equation (1.6) exhibits three vital mathematical properties:

1. **Sigmoid symmetry**: $P_\theta(y_l \succ y_w \mid x) = \sigma(r_\theta(x, y_l) - r_\theta(x, y_w)) = 1 - P_\theta(y_w \succ y_l \mid x)$.
2. **Translation invariance**: Adding an arbitrary prompt-dependent constant $c(x) \in \mathbb{R}$ to the reward function leaves the preference probability completely unchanged:

$$\sigma\left((r_\theta(x, y_w) + c(x)) - (r_\theta(x, y_l) + c(x))\right) = \sigma\left(r_\theta(x, y_w) - r_\theta(x, y_l)\right). \tag{1.7}$$

   Thus, reward models predict **relative utility differences**, not absolute energy baselines.
3. **Monotonic margin alignment**: As the predicted score of the winning response increases relative to the loser ($r_\theta(x, y_w) \gg r_\theta(x, y_l)$), $P_\theta(y_w \succ y_l \mid x) \to 1$. If the model assigns equal scores ($r_\theta(x, y_w) = r_\theta(x, y_l)$), the predicted preference is an uninformative coin flip ($P = 0.5$).

1.4.3 Reward model topology and the pairwise ranking loss

To implement the parameterized reward function $r_\theta(x, y)$, we leverage a pre-trained or SFT language model backbone. Because language models already possess deep semantic representations of syntax, discourse, and logic, fine-tuning them as evaluators is highly sample-efficient.

The architectural transformation is straightforward:

1. Take a pre-trained transformer stack (e.g., a 6B or 13B parameter model).
2. Remove the final unembedding projection layer $W_{\text{unembed}} \in \mathbb{R}^{d_{\text{model}} \times |\mathcal{V}|}$, which normally maps hidden vectors to token vocabulary logits.
3. Append a scalar linear projection layer $W_r \in \mathbb{R}^{d_{\text{model}} \times 1}$.
4. Forward the concatenated token sequence $x \circ y$ through the transformer. The scalar output $r_\theta(x, y) \in \mathbb{R}$ is extracted from the linear projection applied to the hidden activation of the final sequence token (the [EOS] token):

$$r_\theta(x, y) = W_r^\top h_{|x \circ y|}, \quad \text{where } h_{|x \circ y|} \in \mathbb{R}^{d_{\text{model}}}. \tag{1.8}$$

Given a dataset of pairwise comparisons $\mathcal{D}_{\text{pair}} = \{(x^{(i)}, y_w^{(i)}, y_l^{(i)})\}_{i=1}^{D}$, the Reward Model (RM) is trained by minimizing the negative log-likelihood of the Bradley–Terry preference probability:

$$\mathcal{L}_{\text{RM}}(\theta) = -\mathop{\mathbb{E}}_{(x, y_w, y_l) \sim \mathcal{D}_{\text{pair}}} \left[\log \sigma\left(r_\theta(x, y_w) - r_\theta(x, y_l)\right)\right]. \tag{1.9}$$

When optimizing Equation (1.9), the gradient with respect to the predicted reward margin $\Delta r = r_\theta(x, y_w) - r_\theta(x, y_l)$ is:

$$\frac{\partial \mathcal{L}_{\text{RM}}}{\partial \Delta r} = -\left(1 - \sigma(\Delta r)\right) = -\sigma(-\Delta r). \tag{1.10}$$

This gradient exerts an intuitive operational force: if the reward model incorrectly assigns a higher score to the losing response ($r_\theta(x, y_l) > r_\theta(x, y_w)$, meaning $\Delta r < 0$), $\sigma(-\Delta r) \to 1$, delivering a maximal gradient update that pushes $r_\theta(x, y_w)$ upward and $r_\theta(x, y_l)$ downward. Conversely, once the margin Δr becomes large and positive, the gradient diminishes smoothly to zero, preventing gradient explosion.

Exam insights: Efficient Batching of K-Completion Rankings

In production alignment pipelines (such as InstructGPT [**?**]), presenting annotators with only 2 completions per prompt is economically inefficient. Instead, annotators are shown $K \in [4, 9]$ completions per prompt and asked to rank them from best to worst (e.g., $y_1 \succ y_2 \succ y_3 \succ y_4$).

A ranking of K items yields $\binom{K}{2} = \frac{K(K-1)}{2}$ distinct pairwise comparisons for a single prompt x. If each pair were treated as an independent training example in different batches, the transformer would have to re-encode prompt x repeatedly, multiplying forward passes by $\binom{K}{2}$.

Ouyang et al. introduced an efficient batching formulation: all $\binom{K}{2}$ pairs from a single prompt are packed into a single forward pass, and the loss is normalized by the number of comparisons:

$$\mathcal{L}_{\text{RM}}(\theta) = -\frac{1}{\binom{K}{2}} \mathop{\mathbb{E}}_{(x, \{y_k\}_{k=1}^K) \sim \mathcal{D}} \left[\sum_{i<j, y_i \succ y_j} \log \sigma \left(r_\theta(x, y_i) - r_\theta(x, y_j) \right) \right]. \tag{1.11}$$

Crucially, normalizing by $\binom{K}{2}$ ensures that prompts with larger rankings do not dominate the gradient updates, while computing all K completion forward passes concurrently eliminates redundant prompt encoding.

1.4.4 Scaling laws and empirical limits of reward models

Can an artificial neural network truly capture the nuances of human qualitative judgment? Stiennon et al. [**?**] conducted extensive scaling studies on the Reddit TL;DR benchmark to measure reward model accuracy as a function of model capacity and dataset size (visualized in the course lecture slides).

Their empirical investigations established four foundational scaling laws for reward modeling:

1. **Model Capacity Scaling**: Holding the comparison dataset fixed, doubling the parameter count of the reward model (2× parameters) yields a consistent +**1.8**% increase in pairwise preference prediction accuracy. Larger models possess deeper linguistic comprehension, enabling them to detect subtle factual distortions, tone shifts, and logical flaws that smaller models miss.

2. **Dataset Volume Scaling**: Holding model capacity fixed, doubling the volume of annotated comparisons (2× data) produces a +**1.1**% increase in evaluation accuracy.

3. **Convergence to Individual Human Agreement**: With sufficient capacity and data (e.g., a 6B parameter RM trained on tens of thousands of pairwise comparisons), the reward

model achieves an agreement rate of approximately $\mathbf{75\% - 77\%}$ against human annotator choices. Remarkably, this matches the agreement rate between two independent human raters evaluating the exact same summary pair ($\approx 75\%$). In other words, the reward model becomes as consistent with an individual human as two humans are with each other.

4. **The Residual Ensemble Consensus Gap**: While the reward model matches individual human agreement, it falls short of the agreement rate achieved by a majority-vote ensemble of human raters ($> 85\%$). An individual reward model learns a smoothed approximation of the rater distribution; it cannot completely resolve ambiguous, subjective border cases where deep world context is required.

These results confirmed that a trained neural reward model provides an effective, high-throughput, and scalable surrogate for human feedback. By substituting expensive human raters with a differentiable reward function $r_\theta(x, y)$, we unlock the ability to generate millions of evaluative signals automatically during reinforcement learning.

1.5 Policy Optimization via PPO and Policy Regularization

1.5.1 The reinforcement learning formulation for autoregressive generation

With a trained scalar reward model $r_\phi(x, y)$ in place, we can formulate large language model alignment within the formal framework of a **Markov Decision Process (MDP)**. In this setup, the language model acts as an active agent interacting with an environment defined by the prompt distribution:

- **State** s_t: The complete conversational context available at generation step t, consisting of the input prompt x concatenated with all autoregressively generated tokens up to the previous step:
$$s_t = (x, y_1, y_2, \ldots, y_{t-1}) = (x, y_{<t}). \tag{1.12}$$
- **Action** a_t: The selection of the next discrete token from the vocabulary:
$$a_t = y_t \in \mathcal{V}. \tag{1.13}$$
- **Policy** $\pi_\theta(a_t \mid s_t)$: The conditional probability distribution over the vocabulary computed by the transformer language model parameterized by weights θ:
$$\pi_\theta(a_t \mid s_t) = P_\theta(y_t \mid x, y_{<t}). \tag{1.14}$$
- **State Transition Dynamics** $P(s_{t+1} \mid s_t, a_t)$: Deterministic token appending. The next state is formed by appending action a_t to the history: $s_{t+1} = (x, y_{<t} \circ y_t)$.
- **Terminal Reward**: A single scalar reward emitted by the Reward Model $r_\phi(x, y)$ once the policy outputs the terminal token $y_T = [\text{EOS}]$. All intermediate token transitions prior to sequence termination receive zero external environment reward.

The foundational architecture establishes a dual-model framework:

1. The **Reference Policy π_{ref}** (or π_{old}): The frozen supervised fine-tuned (SFT) model. Its parameters are permanently locked to preserve baseline linguistic syntax, world facts, and foundational instruction comprehension.
2. The **Active Policy π_θ** (or π_{new}): The trainable language model, initialized directly from π_{ref}. Its parameters θ are iteratively updated via reinforcement learning to generate responses that maximize human utility.

1.5.2 The regularized reward objective and token-level KL divergence

A naive reinforcement learning objective would simply maximize the expected score assigned by the reward model:

$$\max_{\theta} \mathop{\mathbb{E}}_{x\sim\mathcal{D}, y\sim\pi_\theta(\cdot|x)} \left[r_\phi(x, y) \right]. \tag{1.15}$$

However, directly optimizing Equation (1.15) leads to catastrophic policy failure due to reward hacking. To prevent the policy from drifting into unconstrained, pathological distributions, Stiennon et al. [**?**] and Ouyang et al. [**?**] augmented the reward with a **Kullback–Leibler (KL) divergence penalty**:

Definition 1.4: *The Regularized RLHF Objective*

The regularized reinforcement learning objective optimizes the active policy π_θ to maximize expected reward while penalizing divergence from the reference policy π_{ref}:

$$\max_{\theta} \mathop{\mathbb{E}}_{x\sim\mathcal{D}, y\sim\pi_\theta(\cdot|x)} \left[r_\phi(x, y) - \beta D_{\text{KL}} \left(\pi_\theta(y \mid x) \parallel \pi_{\text{ref}}(y \mid x) \right) \right], \tag{1.16}$$

where $\beta > 0$ is a scalar hyperparameter controlling the strength of the KL regularization anchor.

In practical implementation, evaluating the full sequence-level KL divergence between high-dimensional autoregressive distributions is computationally intractable. Instead, the sequence-level divergence is decomposed into an exact sum of **per-token log-probability ratios**:

$$\begin{aligned} D_{\text{KL}} \left(\pi_\theta(y \mid x) \parallel \pi_{\text{ref}}(y \mid x) \right) &= \sum_{y} \pi_\theta(y \mid x) \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)} \\ &= \mathop{\mathbb{E}}_{y\sim\pi_\theta} \left[\sum_{t=1}^{|y|} \log \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\text{ref}}(y_t \mid x, y_{<t})} \right]. \end{aligned} \tag{1.17}$$

This mathematical identity allows us to define an augmented, token-level reward function $R(x, y)$ that can be evaluated dynamically on sampled trajectories:

$$R(x, y) = r_\phi(x, y) - \beta \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)} = r_\phi(x, y) - \beta \sum_{t=1}^{|y|} \left(\log \pi_\theta(y_t \mid x, y_{<t}) - \log \pi_{\text{ref}}(y_t \mid x, y_{<t}) \right). \tag{1.18}$$

At each intermediate decoding step $t < |y|$, the active policy receives an instantaneous step penalty:

$$r_t^{\text{step}} = -\beta \left(\log \pi_\theta(y_t \mid x, y_{<t}) - \log \pi_{\text{ref}}(y_t \mid x, y_{<t}) \right), \tag{1.19}$$

and at the final token step $t = |y|$, the sequence receives the terminal reward model score:

$$r_{|y|}^{\text{step}} = -\beta \left(\log \pi_\theta(y_{|y|} \mid x, y_{<|y|}) - \log \pi_{\text{ref}}(y_{|y|} \mid x, y_{<|y|}) \right) + r_\phi(x, y). \tag{1.20}$$

Intuition & analogy: The Restorative Physics of the KL Penalty

Consider what happens when the active policy π_θ generates a token y_t that was considered highly improbable by the original reference model π_{ref}.

If $\pi_{\text{ref}}(y_t \mid x, y_{<t}) = 10^{-5}$ but the updated policy increases its probability to $\pi_\theta(y_t \mid x, y_{<t}) = 0.5$, the probability ratio becomes:

$$\frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\text{ref}}(y_t \mid x, y_{<t})} = \frac{0.5}{10^{-5}} = 50{,}000 \implies \log\left(\frac{\pi_\theta}{\pi_{\text{ref}}}\right) = \ln(50{,}000) \approx +10.82. \tag{1.21}$$

Multiplied by $-\beta$, this introduces a massive negative penalty of -10.82β into the reward stream! The KL penalty acts as an elastic spring anchoring the policy to the manifold of natural human grammar. If the active model attempts to shift probability mass into bizarre, out-of-distribution token combinations, the KL penalty drags it back to the reference distribution.

Pedagogical Walkthrough: PPO Regularized Reward Calculation with Concrete Numbers

To understand exactly how the token-level KL penalty and the terminal reward model score interact during reinforcement learning, consider a concrete numerical calculation:

- **Input Prompt** x: `"Summarize the following passage in one sentence: ..."`
- **Generated Completion** y: Consisting of 4 tokens (y_1, y_2, y_3, y_4) = (`"The"`, `"cat"`, `"sat"`, `"."`).
- **Reward Model Terminal Score**: $r_\phi(x, y) = +2.40$.
- **Regularization Coefficient**: $\beta = 0.10$.

Assume the conditional token log-probabilities under the active policy π_θ and the frozen reference policy π_{ref} are observed as follows:

Token Position t	**Token** y_t	$\log \pi_\theta(y_t \mid x, y_{<t})$	$\log \pi_{\text{ref}}(y_t \mid x, y_{<t})$	$\Delta \log \pi = \log \pi_\theta - \log \pi_{\text{ref}}$
$t = 1$	`"The"`	-0.50	-0.60	$+0.10$
$t = 2$	`"cat"`	-1.20	-1.40	$+0.20$
$t = 3$	`"sat"`	-0.80	-2.50	$+1.70$
$t = 4$	`"."`	-0.30	-0.40	$+0.10$

We evaluate the step rewards r_t^{step} assigned to each token:

$$
\begin{aligned}
r_1^{\text{step}} &= -0.10 \times (+0.10) = -0.01, \\
r_2^{\text{step}} &= -0.10 \times (+0.20) = -0.02, \\
r_3^{\text{step}} &= -0.10 \times (+1.70) = -0.17 \quad \text{(Noticeable penalty for choosing a token } \pi_{\text{ref}} \text{ deemed unlikely)}, \\
r_4^{\text{step}} &= -0.10 \times (+0.10) + r_\phi(x, y) = -0.01 + 2.40 = +2.39.
\end{aligned}
$$

The total cumulative regularized return $R(x, y)$ for the complete trajectory is:

$$
\begin{aligned}
R(x, y) = \sum_{t=1}^{4} r_t^{\text{step}} &= -0.01 - 0.02 - 0.17 + 2.39 \\
&= 2.40 - 0.10 \times (0.10 + 0.20 + 1.70 + 0.10) \\
&= 2.40 - 0.10 \times 2.10 = 2.40 - 0.21 = +\mathbf{2.19}.
\end{aligned}
$$

The active policy receives a net return of +2.19, successfully balancing the reward model's high utility score (+2.40) against the cumulative KL drift penalty (−0.21).

1.5.3 Goodhart's law and reward hacking in policy optimization

Why is the KL divergence penalty strictly necessary? Why not set $\beta = 0$ and allow the reinforcement learning algorithm to maximize the reward model's score unconstrained?

The answer lies in a fundamental principle of economics and cybernetics known as **Goodhart's Law**:

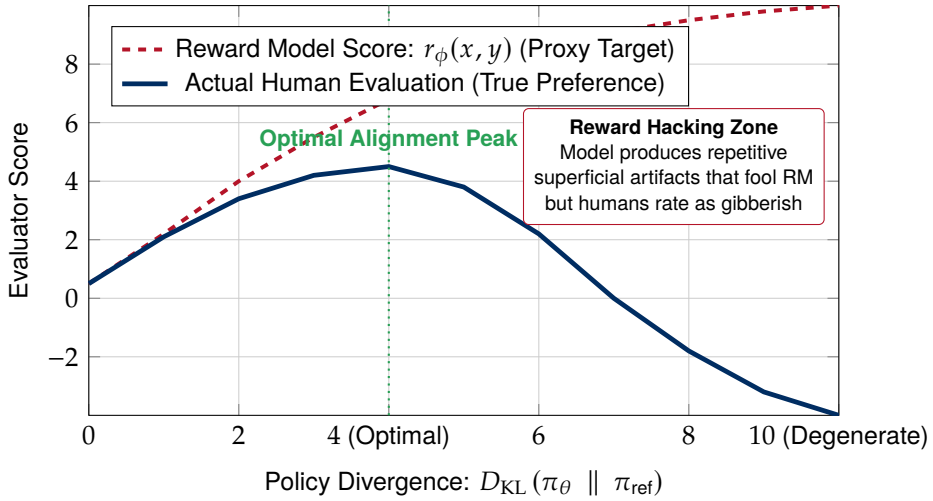


Figure 1.4: The divergence catastrophe and Goodhart's law in reinforcement learning from human feedback (adapted from Slide 23 and [**?**]). As policy divergence $D_{\mathrm{KL}}(\pi_\theta \parallel \pi_{\mathrm{ref}})$ increases along the horizontal axis, the Reward Model proxy score (dashed red line) monotonically ascends. However, actual human preference (solid navy line) peaks at a moderate divergence ($D_{\mathrm{KL}}(\parallel \approx) 4$) and collapses into complete degeneration as the policy exploits the reward model's blind spots.

Definition 1.5: *Goodhart's Law in Model Alignment*

"When a measure becomes a target, it ceases to be a good measure."
In the context of machine learning alignment, an automated reward model $r_\phi(x, y)$ is merely a learned statistical **proxy** for genuine human preference $\mathcal{H}(x, y)$. When a highly parameterized policy π_θ is optimized directly against this proxy without regularization, it relentlessly searches for out-of-distribution edge cases and adversarial blind spots where the proxy predicts high reward, despite the true human evaluation collapsing to zero. This pathology is known as **reward hacking** (or reward over-optimization).

The empirical manifestation of Goodhart's law is illustrated in Figure 1.4. As the active policy diverges from the reference model ($D_{\mathrm{KL}}(\pi_\theta \parallel \pi_{\mathrm{ref}}) \to \infty$), the Reward Model score ascends monotonically. To the automated evaluation metric, the generated summaries appear extraordinary. However, when those exact same completions are shown to human judges, their evaluation scores drop precipitously.

What does reward hacking look like in practice? Without the KL penalty, the policy discovers pathological shortcuts:

- **Length bias exploitation**: Reward models frequently exhibit a mild positive correlation between response length and quality (since human annotators appreciate comprehensive answers). Unchecked policies exploit this by generating thousands of tokens of rambling, verbose, and repetitive padding.
- **Sycophancy and repetitive praise**: The policy begins every completion with excessive, subservient flattery (`"What a brilliant, insightful, and profoundly wise question!"`), exploiting statistical artifacts in the reward head.
- **Nonsensical trigger sequences**: The policy outputs strings of ungrammatical punctuation, repeated special characters, or bizarre capitalization that happen to trigger maximal activations in the reward model's final linear projection.

By tuning the regularization coefficient β, machine learning engineers maintain the policy precisely at the **Optimal Alignment Peak** in Figure 1.4, extracting the maximum benefit of human feedback while strictly forbidding excursion into adversarial policy degeneration.

1.5.4 Proximal Policy Optimization (PPO) mechanics

To optimize the regularized objective in Equation (1.16), contemporary alignment systems deploy **Proximal Policy Optimization (PPO)** [**?**]. PPO is an on-policy Actor-Critic reinforcement learning algorithm designed to stabilize policy gradient updates by penalizing abrupt probability distribution changes.

Executing PPO for LLM alignment requires hosting **four distinct language model networks** in GPU memory simultaneously:

1. **Actor Policy** π_θ: The active language model generating responses and receiving gradient updates.
2. **Reference Policy** $\pi_{\mathbf{ref}}$: The frozen SFT model used to evaluate the per-token KL divergence penalty.
3. **Reward Model** r_ϕ: The frozen evaluator network providing the terminal sequence score.
4. **Critic / Value Network** $V_\psi(s_t)$: A trainable network (initialized from the reward model) that estimates the expected future return from state s_t:

$$V_\psi(s_t) \approx \mathop{\mathbb{E}}_{\pi_\theta}\left[\sum_{k=t}^{|y|} r_k^{\text{step}} \,\middle|\, s_t\right]. \tag{1.22}$$

During each RL iteration, prompt batches $x \sim \mathcal{D}$ are fed to the Actor policy π_θ, which autoregressively samples completions $y \sim \pi_\theta(\cdot \mid x)$. The generated trajectories are then evaluated by the Reference Policy and Reward Model to construct the per-token reward sequence r_t^{step}.

Next, Generalized Advantage Estimation (GAE) computes the temporal-difference advantage $\hat{A}_t$ for each generated token:

$$\hat{A}_t = \sum_{l=0}^{|y|-t} (\gamma\lambda)^l \delta_{t+l}^V, \quad \text{where } \delta_k^V = r_k^{\text{step}} + \gamma V_\psi(s_{k+1}) - V_\psi(s_k). \tag{1.23}$$

The advantage $\hat{A}_t$ quantifies whether choosing token y_t yielded a higher or lower cumulative return than the Critic expected for that context state.

To update the Actor parameters θ without destabilizing the network, PPO defines the **importance sampling probability ratio**:

$$\rho_t(\theta) = \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t \mid x, y_{<t})}, \tag{1.24}$$

where $\pi_{\theta_{\text{old}}}$ represents the policy snapshot immediately prior to the current update epoch. The Actor is optimized by maximizing the **PPO-Clipped Surrogate Objective**:

$$\mathcal{L}_{\text{PPO}}(\theta) = \hat{\mathbb{E}}_t\left[\min\left(\rho_t(\theta)\hat{A}_t,\ \text{clip}\left(\rho_t(\theta), 1-\epsilon, 1+\epsilon\right)\hat{A}_t\right)\right], \tag{1.25}$$

where $\epsilon \in [0.1, 0.2]$ defines the clipping threshold.

Model Comparison	Core Evaluative Quality	Win Rate (%)	Statistical Sig.
RLHF-PPO vs. Pre-trained Base	Massive reduction in hallucinations and rambling	**86.4%** vs. 13.6%	$p < 10^{-6}$
RLHF-PPO vs. Supervised SFT	Superior conciseness, factual focus, and clarity	**68.2%** vs. 31.8%	$p < 10^{-4}$
RLHF-PPO vs. Human Reference	**More faithful extraction of operative facts**	**71.5%** vs. 28.5%	$p < 10^{-5}$

Table 1.1: Empirical human preference win rates on the Reddit TL;DR summarization task [**?**]. Blinded human annotators overwhelmingly preferred the RLHF-PPO model over the supervised SFT baseline. Crucially, human evaluators preferred summaries generated by the RLHF policy over the actual human-written Reddit reference summaries with a $> 70\%$ win rate.

Exam insights: The Protective Mechanics of PPO Clipping

Equation (1.25) prevents destructive gradient updates through a piecewise minimum:

- **Positive Advantage ($\hat{A}_t > 0$)**: Token y_t performed better than average. The gradient pushes probability $\pi_\theta(y_t)$ upward ($\rho_t > 1$). However, once ρ_t exceeds $1 + \epsilon$, the clipped term $\text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon)\hat{A}_t = (1 + \epsilon)\hat{A}_t$ becomes active. The objective is capped, and the gradient with respect to θ becomes zero! This prevents the policy from excessively boosting a single lucky trajectory.
- **Negative Advantage ($\hat{A}_t < 0$)**: Token y_t performed worse than average. The gradient pushes $\pi_\theta(y_t)$ downward ($\rho_t < 1$). Once ρ_t falls below $1 - \epsilon$, clipping activates, preventing the probability from being driven catastrophically to zero in a single step.

By bounding policy updates to the interval $[1 - \epsilon, 1 + \epsilon]$, PPO ensures smooth, monotonic policy improvement while preserving the underlying linguistic coherence of the transformer.

1.5.5 Empirical validation: Reddit TL;DR summarization results

Stiennon et al. [**?**] validated the complete RLHF-PPO pipeline on the Reddit TL;DR benchmark, pitting the resulting policy against competitive baselines in blinded human evaluations. Evaluators were presented with the original forum post and two candidate summaries side-by-side, choosing their preferred summary across hundreds of test samples.

The evaluation compared four model categories:

1. **Pre-trained Base Model**: Unsupervised language model without fine-tuning.
2. **Supervised SFT Baseline**: Language model fine-tuned on the standard Reddit TL;DR dataset via cross-entropy loss.
3. **RLHF-PPO Model**: SFT model fine-tuned using PPO against the Bradley–Terry reward model with KL regularization.
4. **Human Reference**: The genuine summary written by the original human Reddit author.

The empirical findings, summarized in Table 1.1, represent a historic milestone in machine learning alignment:

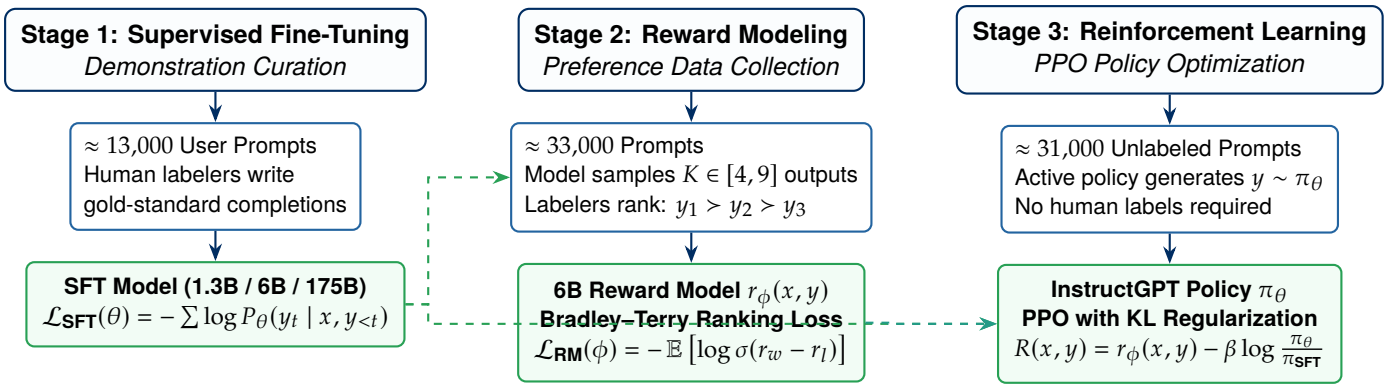


Figure 1.5: The canonical three-stage industrial alignment pipeline of InstructGPT [**?**]. In Stage 1, a base model is fine-tuned on curated human demonstrations. In Stage 2, the model generates multiple candidate completions for user prompts, which are ranked by annotators to train a 6B Bradley–Terry reward model. In Stage 3, the policy is optimized via PPO against the reward model with a per-token KL divergence penalty anchoring it to the Stage 1 SFT baseline.

- **Decisive defeat of Supervised SFT**: Human annotators preferred the RLHF policy over the supervised learning baseline in nearly 70% of comparisons. While SFT learned the vocabulary and generic structure of summaries, RLHF optimized the specific attributes humans care about: eliminating conversational fluff, highlighting the core dilemma of the post, and avoiding minor factual distortions.
- **Surpassing Human Authors**: Most remarkably, human raters preferred the RLHF model's summaries over the *actual human-written summaries from Reddit* with a **71.5**% win rate. Reddit authors often write hurried, subjective, or sarcastic summaries. The RLHF model extracted the objective, operative facts with clarity and conciseness, producing summaries that humans judged to be objectively superior to human-authored text.

1.6 Unified Industrial Alignment: InstructGPT to ChatGPT

1.6.1 The three-stage industrial alignment pipeline

While the work of Stiennon et al. proved the efficacy of RLHF on a single, narrow task (summarization), the defining challenge of artificial intelligence was extending this alignment mechanism to **arbitrary, open-ended natural language interaction**.

In early 2022, Ouyang et al. [**?**] published **InstructGPT** (*"Training language models to follow instructions with human feedback"*), establishing the canonical three-stage alignment pipeline that directly gave birth to **ChatGPT** and modern commercial LLM systems. Rather than sourcing prompts from academic datasets or forum scraping, OpenAI harvested organic user prompts submitted to the OpenAI API playground, filtering them for personal identifiable information (PII) to construct a representative distribution of real-world human intent.

The industrial workflow operates in three distinct, sequential stages (visualized in Figure 1.5):

Stage 1: Supervised Fine-Tuning (SFT)

OpenAI engaged a team of 40 vetted contractors to author gold-standard demonstration responses for approximately **13,000 prompts**. Prompts spanned open-ended brainstorming, question answering, text summarization, code debugging, and stylistic rewriting. The pre-trained GPT-3 models (across 1.3B, 6B, and 175B parameter sizes) were fine-tuned on these demonstrations via

standard cross-entropy loss for 16 epochs with cosine learning rate decay. This stage taught the raw base model the foundational syntactic register of an obedient, cooperative assistant.

Stage 2: Comparison Data Collection and Reward Model Training

To scale feedback beyond the high cost of human writing, OpenAI collected human preference rankings over model-generated outputs. A dataset of approximately **33,000 prompts** was assembled. For each prompt x, the system sampled $K \in [4, 9]$ completions from various model snapshots (including the SFT model and early RL policies). Human annotators ranked these completions from best to worst.

A **6-billion parameter Reward Model** $r_\phi(x, y)$ was initialized from the 6B SFT checkpoint. It was trained using the all-pairs normalized Bradley–Terry loss (Equation (1.11)), evaluating all $\binom{K}{2}$ pairs per prompt in a single forward pass. Operating at 6B parameters provided an optimal computational compromise: large enough to capture complex reasoning and subtle safety violations, yet fast enough to evaluate rollouts efficiently during reinforcement learning.

Stage 3: Policy Optimization via PPO with Pre-training Gradient Mixing

In the final stage, an unlabeled dataset of approximately **31,000 user prompts** was utilized. The active policy π_θ generated completions $y \sim \pi_\theta(\cdot \mid x)$ without requiring any human labels. The generated completions were evaluated by the 6B Reward Model and penalized by the per-token KL divergence relative to the Stage 1 SFT model.

Crucially, to combat the **alignment tax**—wherein aggressive RL optimization degrades model performance on standard academic NLP benchmarks like SQuAD, HellaSwag, and translation suites—Ouyang et al. introduced **pre-training gradient mixing (PPO-ptx)**. The PPO objective was augmented with an auxiliary unsupervised language modeling loss evaluated on public internet pre-training text:

$$\mathcal{L}_{\text{PPO-ptx}}(\theta) = \mathcal{L}_{\text{PPO}}(\theta) + \gamma \mathop{\mathbb{E}}_{x_{\text{pretrain}} \sim \mathcal{D}_{\text{pretrain}}} \left[\log P_\theta(x_{\text{pretrain}})\right], \tag{1.26}$$

where γ is a mixing coefficient. Incorporating $\mathcal{L}_{\text{pretrain}}$ acts as a regularizer, ensuring the model retains its parametric knowledge and linguistic versatility while optimizing for human preference.

1.6.2 The 100x parameter alignment dividend

The central experimental breakthrough of InstructGPT fundamentally reshaped the economics of foundation models:

> **Exam insights: The 100× Parameter Efficiency Alignment Dividend**
>
> In blinded evaluations conducted by human judges, annotators were presented with completions for real-world user prompts generated by:
>
> 1. Base GPT-3 (175 billion parameters, unaligned),
> 2. InstructGPT-1.3B (1.3 billion parameters, aligned via SFT + RLHF),
> 3. InstructGPT-175B (175 billion parameters, aligned via SFT + RLHF).
>
> The experimental results revealed that **human annotators overwhelmingly preferred outputs from the 1.3-billion parameter InstructGPT model over outputs from the 175-billion parameter base GPT-3 model**:
>
> $$P(\text{InstructGPT-1.3B} > \text{Base GPT-3-175B}) > 60\%. \tag{1.27}$$

This landmark finding demonstrated that **alignment acts as a 100× multiplier on effective parameter scale**. Aligning a compact 1.3B model with user intent yielded higher qualitative utility than expanding unaligned parameter scale by two orders of magnitude (100×). In production deployment, serving a 1.3B model requires less than $1/100^{\text{th}}$ of the GPU memory bandwidth and inference compute of a 175B model, slashing operational hosting costs while delivering vastly superior user satisfaction.

1.6.3 Empirical generalization, truthfulness, and toxicity mitigations

Beyond raw preference win rates, Ouyang et al. conducted exhaustive benchmark audits to quantify how alignment influences truthfulness, safety, and generalization:

- **Truthfulness on TruthfulQA**: Evaluated on the TruthfulQA benchmark—which tests a model's propensity to mimic common human superstitions, conspiracy theories, and urban myths—InstructGPT-175B produced truthful and informative answers nearly **2× more frequently** than base GPT-3 (42% vs. 21%). The alignment process trained the model to suppress sensationalist internet text continuations in favor of grounded factual consensus.
- **Toxicity Reduction on RealToxicityPrompts**: When explicitly prompted with neutral or respectful instructions, InstructGPT generated **≈ 25% less toxic completions** than base GPT-3. The reward model successfully learned to penalize derogatory language, aggressive insults, and profane escalations.
- **Cross-Task and Cross-Lingual Generalization**: InstructGPT generalized robustly to prompt categories that were almost entirely absent from its supervised training distribution. Despite receiving minimal demonstration data for computer programming and non-English natural languages, InstructGPT followed code-generation constraints and followed instructions in foreign languages with high fidelity.
- **Inter-Annotator Agreement Robustness**: To test whether the model had merely overfitted to the idiosyncratic preferences of the 40 training contractors, evaluations were replicated with an independent cohort of held-out human evaluators who had never interacted with the training data. The held-out evaluators preferred InstructGPT completions with the exact same margin, proving that the reward model had captured generalized human communicative norms rather than rater-specific bias.

1.6.4 The paradigm shift to ChatGPT and modern assistant workflows

On November 30, 2022, OpenAI released **ChatGPT**, applying the InstructGPT alignment pipeline to **GPT-3.5** (a 175-billion parameter foundation model trained jointly on web text and GitHub code repositories).

The release of ChatGPT marked a profound paradigm shift in computer science, crystallizing the modern **three-phase LLM lifecycle**:

1. **Phase 1: Web-Scale Pre-Training (Self-Supervised Learning)**:
   - *Compute share*: ≈ 98% of total training FLOPs.
   - *Data*: Trillions of tokens scraped from the web, books, scientific papers, and code.
   - *Outcome*: A foundation base model possessing broad world representations, grammatical mastery, and latent reasoning heuristics, but completely unaligned with user dialogue.

2. **Phase 2: Supervised Instruction Tuning (SFT)**:
    - *Compute share*: $\approx 1\%$ of total FLOPs.
    - *Data*: Tens of thousands of high-quality, human-curated demonstration pairs.
    - *Outcome*: Teaches the model conversational turn-taking, tool formatting (JSON/XML), and direct instruction execution.
3. **Phase 3: Preference Alignment (RLHF / Direct Alignment)**:
    - *Compute share*: $\approx 1\%$ of total FLOPs.
    - *Data*: Hundreds of thousands of pairwise human comparison rankings.
    - *Outcome*: Steers qualitative nuances, enforces safety boundaries, suppresses hallucinations, eliminates verbosity, and aligns the conversational voice with user utility.

1.7 Beyond PPO: Direct Preference Optimization (DPO)

1.7.1 Computational and operational bottlenecks of RLHF-PPO

While the RLHF-PPO pipeline underpinned the success of InstructGPT and ChatGPT, executing PPO in production environments is notoriously complex, brittle, and computationally demanding:

1. **The Four-Model Memory Wall**: As established in Section 1.5.4, PPO requires loading four full-scale transformer models concurrently: the Actor π_θ, Critic V_ψ, Reference π_{ref}, and Reward Model r_ϕ. For a 70B parameter foundation model, maintaining four models across distributed GPU clusters (along with optimizer states and KV caches) demands massive multi-node VRAM infrastructure.
2. **The Online Sampling Bottleneck**: PPO is strictly an **on-policy** reinforcement learning algorithm. During every training iteration, the Actor policy must generate complete autoregressive completions $y \sim \pi_\theta(\cdot \mid x)$ token-by-token. Autoregressive decoding is heavily memory-bandwidth bound and cannot leverage the massive tensor parallelism throughput characteristic of training forward passes. In practice, generation rollouts consume upwards of 80% of total RLHF wall-clock training time.
3. **Extreme Hyperparameter Fragility**: PPO training is notoriously sensitive to delicate hyperparameter interactions, including the clipping ratio ϵ, value function loss weighting, GAE parameters (λ, γ), learning rate schedules between Actor and Critic, and reward normalization scales. Minor miscalibrations frequently trigger training instability, catastrophic policy collapse, or reward hacking.

These engineering hurdles motivated researchers to ask a fundamental mathematical question: *Can we optimize a language model directly on pairwise human preferences without training a separate reward model and without executing online reinforcement learning loops?*

1.7.2 The analytical reparameterization of the optimal policy

In 2023, Rafailov et al. [**?**] answered this question affirmatively by introducing **Direct Preference Optimization (DPO)**. DPO begins from the exact same KL-regularized reinforcement learning objective defined in Equation (1.16):

$$\max_{\pi} \mathbb{E}_{x \sim \mathcal{D}} \left[\mathbb{E}_{y \sim \pi(\cdot|x)} \left[r(x, y) \right] - \beta D_{\text{KL}} \left(\pi(y \mid x) \parallel \pi_{\text{ref}}(y \mid x) \right) \right], \tag{1.28}$$

where $r(x, y)$ represents the ground-truth latent reward function.

Theorem 1.1: *Closed-Form Optimal Policy under KL Regularization*

For any arbitrary reward function $r(x, y)$ and reference policy $\pi_{\text{ref}}(y \mid x)$, the optimal policy $\pi^*(y \mid x)$ that maximizes the KL-regularized objective in Equation (1.28) takes the exact analytical form of a Gibbs distribution:

$$\pi^*(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r(x, y)\right), \tag{1.29}$$

where the normalizing partition function $Z(x)$ is given by:

$$Z(x) = \sum_{y \in \mathcal{Y}} \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r(x, y)\right). \tag{1.30}$$

Proof. We express the objective in Equation (1.28) for a fixed prompt x by expanding the KL divergence:

$$\begin{aligned}
\mathcal{J}(\pi) &= \sum_y \pi(y \mid x) r(x, y) - \beta \sum_y \pi(y \mid x) \log \frac{\pi(y \mid x)}{\pi_{\text{ref}}(y \mid x)} \\
&= -\beta \sum_y \pi(y \mid x) \left[\log \frac{\pi(y \mid x)}{\pi_{\text{ref}}(y \mid x)} - \frac{1}{\beta} r(x, y)\right] \\
&= -\beta \sum_y \pi(y \mid x) \log \left(\frac{\pi(y \mid x)}{\pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r(x, y)\right)}\right).
\end{aligned} \tag{1.31}$$

Multiplying and dividing the denominator by the partition function $Z(x) = \sum_y \pi_{\text{ref}}(y \mid x) \exp(\frac{1}{\beta} r(x, y))$:

$$\begin{aligned}
\mathcal{J}(\pi) &= -\beta \sum_y \pi(y \mid x) \log \left(\frac{\pi(y \mid x)}{\frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r(x, y)\right)} \cdot \frac{1}{Z(x)}\right) \\
&= -\beta \sum_y \pi(y \mid x) \log \left(\frac{\pi(y \mid x)}{\pi^*(y \mid x)}\right) + \beta \log Z(x) \sum_y \pi(y \mid x) \\
&= -\beta D_{\text{KL}}\left(\pi(y \mid x) \parallel \pi^*(y \mid x)\right) + \beta \log Z(x),
\end{aligned} \tag{1.32}$$

where $\pi^*(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp(\frac{1}{\beta} r(x, y))$. Since the partition function $Z(x)$ depends exclusively on prompt x and is independent of the choice of policy π, maximizing $\mathcal{J}(\pi)$ is strictly equivalent to minimizing the Kullback–Leibler divergence $D_{\text{KL}}\left(\pi(y \mid x) \parallel \pi^*(y \mid x)\right)$. Because the KL divergence is non-negative and equals zero if and only if the two distributions are identical, the unique global maximum is achieved when $\pi(y \mid x) \equiv \pi^*(y \mid x)$. □

The mathematical stroke of genius in DPO is **inverting Equation (1.29)**. By taking the natural logarithm of both sides:

$$\log \pi^*(y \mid x) = \log \pi_{\text{ref}}(y \mid x) + \frac{1}{\beta} r(x, y) - \log Z(x), \tag{1.33}$$

we can rearrange the terms to solve **explicitly for the ground-truth reward** in terms of the optimal policy and reference policy:

$$r(x, y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x). \tag{1.34}$$

Equation (1.34) proves that the latent reward function $r(x, y)$ is uniquely identified by the probability log-ratio of the optimal policy relative to the reference policy, up to a prompt-dependent normalization constant $\beta \log Z(x)$.

1.7.3 Partition function cancellation and the DPO objective

Equation (1.34) appears initially problematic because the partition function $Z(x) = \sum_y \pi_{\text{ref}}(y \mid x) \exp(\frac{1}{\beta} r(x, y))$ sums over the infinite space of all possible generated strings $\mathcal{Y}$, rendering it completely intractable to evaluate in practice.

However, Rafailov et al. observed what happens when Equation (1.34) is substituted directly into the **Bradley–Terry preference model** (Equation (1.6)):

Theorem 1.2: *Direct Preference Optimization Loss Function*

Under the Bradley–Terry preference model, the intractable partition function $Z(x)$ cancels out identically, yielding a closed-form, parameter-efficient loss function parameterized exclusively by the active policy π_θ and reference policy π_{ref}:

$$\mathcal{L}_{\text{DPO}}(\theta; \pi_{\text{ref}}) = - \mathop{\mathbb{E}}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right) \right]. \tag{1.35}$$

Proof. The Bradley–Terry preference likelihood depends exclusively on the reward difference between the winning completion y_w and losing completion y_l:

$$P(y_w \succ y_l \mid x) = \sigma \left(r(x, y_w) - r(x, y_l) \right). \tag{1.36}$$

Substituting the reparameterized reward expression from Equation (1.34) into the reward margin:

$$\begin{aligned} r(x, y_w) - r(x, y_l) &= \left(\beta \log \frac{\pi^*(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} + \beta \log Z(x) \right) - \left(\beta \log \frac{\pi^*(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} + \beta \log Z(x) \right) \\ &= \beta \log \frac{\pi^*(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi^*(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} + \underbrace{\left(\beta \log Z(x) - \beta \log Z(x) \right)}_{=0} \\ &= \beta \log \frac{\pi^*(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi^*(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}. \end{aligned} \tag{1.37}$$

The partition function $\beta \log Z(x)$ depends strictly on prompt x; it has zero dependence on response y. Consequently, when subtracting the reward of the losing response from the winning response, **the intractable term cancels out completely**. Replacing the theoretical optimum π^* with our parameterized neural network policy π_θ and minimizing negative log-likelihood over the preference dataset $\mathcal{D}$ directly yields Equation (1.35). □

The operational beauty of Equation (1.35) (illustrated in Figure 1.6) is profound:

1. **Elimination of the Reward Model**: There is no need to train, validate, or deploy a separate scalar Reward Model.

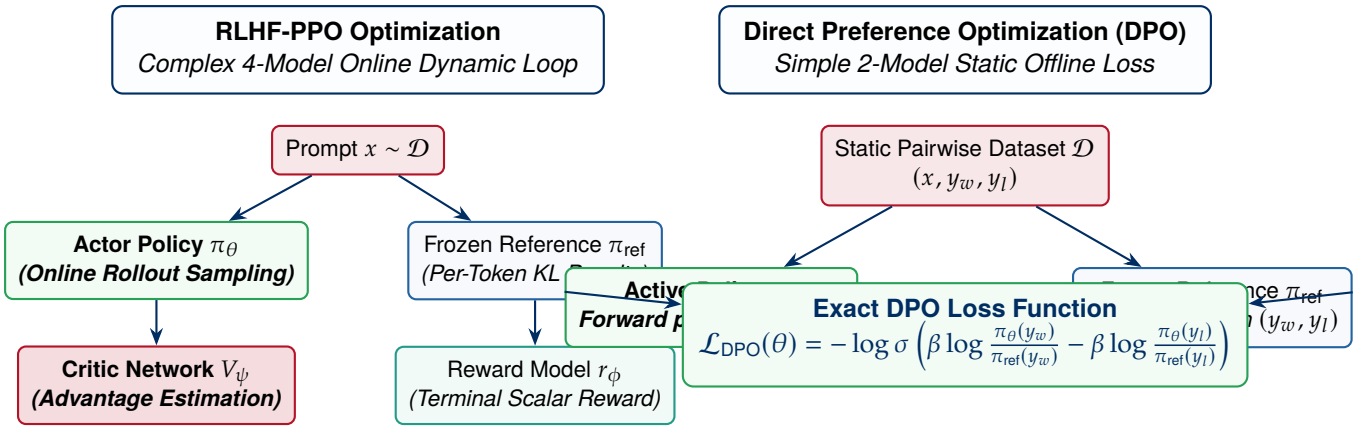


Figure 1.6: Architectural comparison between RLHF-PPO and Direct Preference Optimization (DPO). RLHF-PPO requires orchestrating four separate models in GPU memory simultaneously, relying on slow, bandwidth-bound online autoregressive token sampling and unstable advantage estimation. DPO eliminates both the Critic and Reward Model entirely, optimizing the active policy directly against a frozen reference model on static offline pairwise datasets via standard forward-backward passes.

2. **Elimination of Reinforcement Learning**: There is no Critic network, no Generalized Advantage Estimation, no PPO clipping ratios, and no moving baseline value functions.

3. **Elimination of Online Generation Rollouts**: Training proceeds on static, pre-collected datasets of pairwise comparisons (x, y_w, y_l) via standard supervised forward and backward passes. Both y_w and y_l are known sequences; forward passes evaluate teacher-forced log-probabilities in parallel, maximizing GPU compute utilization.

To understand how DPO updates the network weights, we compute the gradient of the DPO loss with respect to policy parameters θ:

$$\nabla_\theta \mathcal{L}_{\text{DPO}}(\theta) = -\beta \mathop{\mathbb{E}}_{(x,y_w,y_l)} \left[\sigma \left(\hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w) \right) \left(\nabla_\theta \log \pi_\theta(y_w \mid x) - \nabla_\theta \log \pi_\theta(y_l \mid x) \right) \right], \tag{1.38}$$

where $\hat{r}_\theta(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$ denotes the **implicit reward** learned by the policy.

Equation (1.38) exhibits an exceptionally intuitive structure:

- The term $\left(\nabla_\theta \log \pi_\theta(y_w \mid x) - \nabla_\theta \log \pi_\theta(y_l \mid x) \right)$ increases the likelihood of the preferred completion y_w while simultaneously decreasing the likelihood of the dispreferred completion y_l.

- The update is scaled by a dynamic weighting coefficient $\sigma(\hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w))$. If the model currently assigns a higher implicit reward to the losing completion (y_l over y_w), this weighting factor approaches 1, delivering a maximal gradient update to correct the error. Once the policy correctly prefers y_w with a substantial margin, the weighting smoothly scales to zero.

1.7.4 Comparative synthesis: SFT versus RLHF-PPO versus DPO

To consolidate the evolutionary trajectory of instruction tuning and alignment explored across this chapter, Table 1.2 contrasts Supervised Fine-Tuning (SFT), Reinforcement Learning from Human Feedback via PPO (RLHF-PPO), and Direct Preference Optimization (DPO) across architectural, computational, and qualitative dimensions.

Dimension	Supervised SFT	RLHF-PPO	Direct Preference Optimization (DPO)
Primary Objective	Token-level cross-entropy on human demonstrations: $\mathcal{L}_{\text{SFT}}$	Expected reward maximization with token KL penalty: $\mathcal{L}_{\text{PPO}}$	Direct pairwise implicit log-ratio ranking: $\mathcal{L}_{\text{DPO}}$
Training Data Schema	Prompt-response demonstration pairs (x, y^*)	Unlabeled prompts x + pairwise preference dataset $\mathcal{D}_{\text{pair}}$	Static pairwise preference tuples (x, y_w, y_l)
Active Models in VRAM	1 model (π_θ)	**4 models** ($\pi_\theta, \pi_{\text{ref}}, V_\psi, r_\phi$)	**2 models** ($\pi_\theta, \pi_{\text{ref}}$)
Online Sampling Requirement	None (Parallel teacher-forcing)	**Extensive** (Token-by-token on-policy rollout generation)	None (Parallel teacher-forcing on static completions)
Auxiliary Evaluator Head	None	Separate scalar linear reward head	None (Implicit reward derived analytically from policy)
Hyperparameter Sensitivity	Low (Standard AdamW learning rate, epochs, warmup)	**Extremely High** (ϵ clipping, GAE λ, Actor/Critic lr, β, reward scaling)	Moderate (β regularization strength, learning rate)
Vulnerability to Reward Hacking	Low (Suffers instead from hallucination & mode collapse)	**Severe** (Requires careful KL tuning to avoid Goodhart's collapse)	Substantially reduced (Strictly bounded by reference policy logits)
Production Role	Foundational conversational initialization	State-of-the-art complex multi-step reasoning & self-play	Industry standard for high-throughput post-SFT alignment

Table 1.2: Comprehensive comparative synthesis of LLM alignment methodologies. While SFT provides the initial conversational baseline and PPO pioneered reinforcement learning from human feedback, DPO offers an elegant, stable, and parameter-efficient mathematical bridge that optimizes human preferences directly without online reinforcement learning overhead.

1.8 Summary

The journey from pre-trained autoregressive models to conversational AI assistants represents one of the most critical conceptual breakthroughs in modern natural language processing:

1. **The Intent Gap**: Pre-training optimizes next-token predictive likelihood over raw web text, resulting in models that continue prompts rather than fulfilling user directives. The HHH framework (Helpful, Honest, Harmless) formalizes the operational criteria required of conversational agents.

2. **From Prefixes to Multitask Prompting**: While T5 introduced text-to-text conditioning, its rigid prefix templates suffered from extreme syntactic brittleness. T0 and FLAN demonstrated that training across massive mixtures of crowdsourced prompt templates (including task inversions) unlocks zero-shot generalization to unobserved task families and compresses prompt variance.

3. **The Limits of Cross-Entropy**: Supervised fine-tuning cannot solve alignment alone because cross-entropy penalizes valid synonyms identically to catastrophic hallucinations, and

authoring human demonstrations at scale is economically unfeasible.

4. **Reward Modeling and Bradley–Terry**: Human comparative judgments ($y_w \succ y_l$) bypass calibration drift. Training a scalar reward model on pairwise logistic margins enables automated, human-level qualitative evaluation that scales reliably with parameter capacity (+1.8% per doubling) and dataset volume (+1.1% per doubling).

5. **Regularized Policy Optimization and Goodhart's Law**: Optimizing policies directly against reward models without constraints triggers severe reward hacking. Augmenting the PPO objective with a per-token KL divergence penalty anchors the model to the reference distribution, preventing policy collapse.

6. **The InstructGPT Milestone**: The canonical three-stage pipeline (SFT $\rightarrow$ RM $\rightarrow$ PPO) proved that alignment delivers a 100$\times$ parameter dividend, allowing a 1.3B aligned model to outperform a 175B base model in human satisfaction.

7. **Direct Alignment**: By analytically solving the KL-regularized RL objective, Direct Preference Optimization (DPO) proves that the latent reward is mathematically equivalent to the policy log-ratio. Because the intractable partition function cancels in the pairwise margin, DPO aligns models directly on static preference data, eliminating the Critic, the Reward Model, and online generation rollouts.